

Wax Bite Image Processor

Setup & User Guide

macOS and Windows

Contents

1	Overview	2
2	Folder Structure	2
3	One-Time Installation	3
4	Updating the Application	5
5	Running the Application	6
6	Manual Annotation (the manual step)	7
7	Tooth Templates	8
8	Interactive Analysis (the analyze step)	9
9	Troubleshooting	9

Overview

The Wax Bite Image Processor analyzes wax bite print images of gecko teeth taken at multiple time points to track tooth eruption and loss over time. Images are processed through a pipeline of steps, each building on the output of the previous one.

Pipeline Summary

Step	Command	Description
Template Matching	<code>match</code>	Automatically locates teeth in each image
Manual Annotation	<code>manual</code>	Review and correct automated results
Fit & Project	<code>fitproj</code>	Fits a hyperbola to the dental arch; projects to 1D
Format	<code>format</code>	Combines data across time points; generates plots
Analyze	<code>analyze</code>	Opens plots in an interactive exploration window

Which steps are required? `manual` and `fitproj` are the core required steps. `match` is optional but saves time by pre-populating annotations. `format` and `analyze` are run after all images are processed to view results.

Folder Structure

The application is designed to work with one animal's images at a time. A recommended folder structure keeps each animal's data organized outside the repository so the repository stays clean between sessions.

Recommended Layout

```

Documents/
  GeckoData/                                <- your storage folder (name it anything)
    Animals/
      animal1/
        img/                                <- original images live here permanently
        processed/                          <- results moved here after each session
      animal2/
        img/
        processed/
  Wax-Bite-Image-Processor/ <- the repository (cloned once)
    TemplateMatching/
      img/                                  <- working folder: copy images here to process
      processed/                          <- working folder: results appear here
      template/                            <- tooth templates for automated matching
      ...

```

Workflow for Each Animal

Follow these steps each time you want to process a new animal's images. Your originals are never deleted — you only ever work with copies inside the repository.

1. **Copy images into the repository.** Copy all image files from `Animals/animal1/img/` into

Wax-Bite-Image-Processor/TemplateMatching/img/. Your originals remain safely in the Animals folder.

- 2. Restore previous results (if re-analyzing).** If you have previously processed this animal and want to continue from where you left off, copy the contents of `Animals/animal1/processed/` back into `TemplateMatching/processed/`.
- 3. Run the analysis.** Follow the instructions in Sections 3 through 5.
- 4. Move results to the animal's storage folder.** When done, move (not copy) the contents of `TemplateMatching/processed/` into `Animals/animal1/processed/`.
- 5. Delete the image copies from the repository.** Delete all files from `TemplateMatching/img/`. Your originals are untouched. The repository is now clean and ready for the next animal.

Image Naming

Image file names must begin with a date in one of two formats, followed by any additional identifier:

- `YYYY_MM_DD` (preferred) — e.g. `2024_03_15_LG282.jpg`
- `MM_DD_YYYY` (legacy) — e.g. `03_15_2024_LG282.jpg`

The application detects which format is used automatically. The date prefix is used to sort images chronologically and to label results in plots. Images that do not start with a date in one of these formats will cause errors.

Recommendation: Use the `YYYY_MM_DD` format for new data. Files sort chronologically in your file browser, and all new data in the lab follows this convention.

One-Time Installation

macOS

Step 1: Install Homebrew (if not already installed)

Homebrew is a package manager that makes installing developer tools straightforward. Open the Terminal application (find it in Applications → Utilities, or search with Spotlight) and paste:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Follow the on-screen prompts. When it finishes, close and reopen Terminal.

Step 2: Install Python and Git

```
brew install python@3.11 git
```

Step 3: Clone the repository

Navigate to the folder where you want to store the repository (e.g. `~/Documents/GeckoData`), then clone:

```
cd ~/Documents/GeckoData
git clone https://github.com/ecytryn/Wax-Bite-Image-Processor.git
```

```
cd Wax-Bite-Image-Processor/TemplateMatching
```

Step 4: Create and activate a virtual environment

```
python3.11 -m venv venv  
source venv/bin/activate
```

You will see (venv) appear at the start of your terminal prompt when the environment is active.

Step 5: Install required packages

```
pip install -r requirements.txt
```

This may take several minutes. When it finishes and the prompt returns, installation is complete.

Step 6: Initialize the folder structure

```
python main.py
```

This creates the `img/`, `template/`, and `processed/` subfolders.

Verifying the installation: After running `python main.py` for the first time you should see an `img/` folder appear inside `TemplateMatching/`. If you see error messages instead, check the Troubleshooting section.

Windows

Step 1: Install Python

1. Go to <https://www.python.org/downloads/release/python-3110/> in your browser.
2. Scroll down and click **Windows installer (64-bit)** to download.
3. Run the installer. **Important:** before clicking “Install Now”, check the box that says “**Add Python to PATH**” at the bottom of the window. This step is critical. If you have administrator privileges, also check “**Install for all users**” so that everyone who connects to this machine can use the app.
4. Click “Install Now” and wait for it to finish.

To verify Python installed correctly: press the Windows key, type `cmd`, press Enter. In the black window that opens, type `python --version` and press Enter. You should see `Python 3.11.x`.

Step 2: Install Git

1. Go to <https://git-scm.com/download/win> — the download starts automatically.
2. Run the installer, clicking Next through all steps; the defaults are fine.

Step 3: Clone the repository

1. Open File Explorer and navigate to the folder where you want to store the repository (e.g. your Documents folder).

2. Click in the address bar at the top of File Explorer, type `cmd`, and press Enter. A Command Prompt window will open in that folder.
3. Type the following and press Enter:

```
git clone https://github.com/ecytryn/Wax-Bite-Image-Processor.git
cd Wax-Bite-Image-Processor\TemplateMatching
```

Step 4: Create and activate a virtual environment

```
python -m venv venv
venv\Scripts\activate.bat
```

You will see `(venv)` appear at the start of the line when the environment is active.

Step 5: Install required packages

```
pip install -r requirements.txt
```

Step 6: Initialize the folder structure

```
python main.py
```

Every session: You must activate the virtual environment each time you open a new terminal window before running the application. On macOS: `source venv/bin/activate`. On Windows: `venv\Scripts\activate.bat`. If you forget, you will see a `ModuleNotFoundError`.

If the code has been updated and new packages are required, run `pip install -r requirements.txt` after activating the virtual environment. There is no need to recreate the virtual environment itself.

Updating the Application

When the code has been updated, use `git pull` to download the latest version. Open a terminal in the `Wax-Bite-Image-Processor` folder (the top-level repository folder, *not* the `TemplateMatching` subfolder) and run:

```
git pull
```

Then navigate back into the `TemplateMatching` folder, activate the virtual environment, and install any new packages:

macOS:

```
cd TemplateMatching
source venv/bin/activate
pip install -r requirements.txt
```

Windows:

```
cd TemplateMatching
venv\Scripts\activate.bat
pip install -r requirements.txt
```

The `pip install` step is only needed when the update includes new package dependencies. Running it when no new packages are required is harmless — it will simply report that everything is already installed.

Running the Application

All commands are run from inside the `TemplateMatching/` folder with the virtual environment active.

Basic Command Format

```
python main.py [steps] [image selection flags] [display flags]
```

Steps (one or more, space-separated): `match`, `manual`, `fitproj`, `format`, `analyze`

Image selection flags:

Flag	Meaning
<code>-s N</code>	Start at image index N (first image is index 0)
<code>-n N</code>	Process N images

Display flags:

Flag	Meaning
<code>--width N</code>	Set annotation window width to N pixels for this session

The default window width is controlled by `MAX.WIDTH` in `utils.py` (default: 1200 pixels). Use `--width` to override this for a session without editing the file. If the image appears too small or too large, try adjusting this value. A good starting point is roughly half your screen width.

Common Commands

```
# Run the full pipeline on all images
python main.py

# Run only template matching on all images
python main.py match

# Run manual annotation on all images
python main.py manual

# Run manual annotation with a custom window width
python main.py manual --width 1200

# Run fit and project on all images
python main.py fitproj

# Run multiple steps in sequence on all images
python main.py match manual fitproj

# Process only the 3rd image (index 2) through manual annotation
python main.py -s 2 -n 1 manual
```

```
# Process 5 images starting from the 1st
python main.py -s 0 -n 5 match fitproj

# Generate and view plots (no image selection needed)
python main.py format
python main.py analyze

# Process specific named images
python main.py 03_15_2024_LG282.jpg 03_22_2024_LG282.jpg manual
```

Recommended workflow for a new dataset:

1. `python main.py match` — run automated matching on all images
2. `python main.py manual` — manually review and correct each image
3. `python main.py fitproj` — fit hyperbola and project
4. `python main.py analyze` — view results interactively

Running steps separately (rather than all at once) gives you more control and makes it easier to identify which step caused an error.

Manual Annotation (the manual step)

When you run the `manual` step, a window opens showing the current image. If template matching has already been run, those results are pre-loaded as a starting point.

Annotation Modes

There are six types of markers you can place:

Mode	Key	Use for
Tooth	1	A tooth
Gap	2	A gap between teeth
Center Tooth	3	The central-most tooth (only one center allowed)
Center Gap	4	The central-most gap (only one center allowed)
No Bite	5	A position where the gecko did not bite into the wax
Center No Bite	6	The central-most position, in a no-bite zone (only one center allowed)

When to use No Bite: Sometimes the gecko does not bite fully into the wax, leaving a region with no visible tooth or gap imprints. Use the No Bite marker (5) to indicate where teeth or gaps *should* be but cannot be determined from the wax impression. It is important to place the correct number of No Bite markers in these zones so that teeth distal to the zone stay properly aligned across time points. Compare with neighboring rows in the index or arclength plot to verify the count. If the center of the jaw falls in a no-bite zone, use Center No Bite (6) for that position.

Keyboard Shortcuts

Key	Action
1 / 2 / 3 / 4 / 5 / 6	Switch mode (tooth / gap / center tooth / center gap / no bite / center no bite)
Tab	Cycle through modes in order
Space	Toggle boxes on/off (view image without overlays)
S	Save and advance to the next image
X	Save and stop (exit the manual loop entirely)
Q	Quit without saving
Esc	Close current image without saving
← arrow	Save and go to previous image
→ arrow	Save and go to next image

Mouse Controls

- **Left click** on empty space: place a marker of the current mode
- **Left click** inside an existing box: delete that marker

Always save before closing using S, X, or the arrow keys. Q and Esc exit without saving the *current* image, which will cause `fitproj` to fail for that image if no previous annotation exists for it. Images you navigated through using the arrow keys are saved automatically and are not affected.

Stopping early: If you only need to annotate a few images, press X when you are done. This saves the current image and exits without opening the remaining images. S and the arrow keys save but continue to the next image.

Window size: The annotation window opens at the width set by `MAX_WIDTH` in `utils.py` (default: 1200 pixels), or the value passed via `--width` on the command line. If the image appears too small, increase this value; if it doesn't fit on your screen, decrease it. Avoid resizing the window manually by dragging its edges, as this will cause click positions to be misaligned with markers.

Tooth Templates

The `template/` folder contains small image patches of individual teeth used by the `match` step to automatically locate teeth in each image. The current templates were created from images produced with the standard staining protocol used in the lab.

When to add templates

If you notice that a consistent type or shape of tooth is frequently missed by the automated matching, adding one or more example images of that tooth to the `template/` folder will help the matcher find it. Templates should be:

- Small crops of a single tooth (not multiple teeth)
- Representative of the appearance you want to detect
- Saved as `.jpg` or `.png`

Using unstained images

If you are working with unstained wax bites, the existing templates are unlikely to produce good results since tooth appearance will differ significantly. In this case, create a new set of templates from your unstained images and either replace or supplement the existing ones in the `template/` folder. The `manual` step can always be used to correct or replace the automated results regardless of template quality.

Interactive Analysis (the analyze step)

Running `python main.py analyze` generates four plots and opens them in interactive windows:

- **Arclength plot** — tooth positions in arclength coordinates over time
- **Index plot** — tooth positions by index over time
- **Eruption plot** — eruption events colored by time since last eruption
- **Fall-out plot** — tooth loss events colored by time since last loss

Interacting with the plots

- **Left click** on a point: opens the projected 1D image for that date. Each tooth position is labeled with its index number (relative to the center marker, which is index 0).
- **Left click and drag** across a range of dates: opens the projected 1D images for all dates in the selected range, stacked vertically and aligned to the same width, with index labels on each strip. This is useful for visually comparing annotations across nearby days and identifying errors such as a tooth misidentified as a gap, or a missed gap causing all teeth distal to it to appear shifted relative to other days.
- **Right click** (ctrl+click on Mac): opens two windows side by side — the manual annotation tool for that date, and a reference window showing the labeled 1D strips for that date and its two nearest neighbors. Both views show tooth index numbers, making it easy to identify a problematic tooth in the 1D reference and locate it in the 2D editor. The reference window closes automatically when you close the editor. Note: changes will not appear in the plots until you re-run `format` or `analyze`.

The matplotlib plot windows also have a built-in toolbar that allows you to zoom and pan within the plots. Look for the toolbar at the bottom of each plot window.

Troubleshooting

ModuleNotFoundError when running the app

The virtual environment is not activated. Make sure you see `(venv)` at the start of your command line. macOS: `source venv/bin/activate` Windows: `venv\Scripts\activate.bat`

AssertionError: manual data.csv does not exist

The `fitproj` step cannot find annotation data for one or more images. This usually means the `manual` step was exited with `Esc` or `Q` rather than saved with `S` or the arrow keys. Re-run `python main.py manual` and save each image with `S`.

python not recognized (Windows)

Python was not added to `PATH` during installation. Uninstall Python and reinstall, making sure to check “Add Python to `PATH`” at the bottom of the installer window.

Image appears too small or too large

Use the `--width` flag to set a comfortable window size, for example: `python main.py manual --width 1400`. A good starting point is roughly half your screen width. Once you find a value that works, update `MAX.WIDTH` in `utils.py` (currently 1200) to make it the default.

Click positions are misaligned with markers

This can happen if the annotation window has been manually resized by dragging its edges. Close the window and reopen it without resizing. If the problem persists, try a different `--width` value. This issue is particularly common over remote desktop connections.

Python not found when connecting remotely (Windows)

Python may have been installed for a specific user account rather than for all users. Ask an administrator to reinstall Python with the “Install for all users” option checked.

RuntimeError: Unable to fit a Hyperbola or Parabola

The `fitproj` step detected a circle or ellipse instead of a hyperbola. This is usually caused by annotation errors. Open the image in the `manual` step, check for misplaced markers (especially any that are far from the dental arch), correct them, save, and re-run `fitproj`.

A window appears and immediately closes

Look at the terminal for an error message. If you see a traceback (lines beginning with `File "..."`), note the last few lines — they identify the specific error. Refer to this guide or contact the developer.

Plots are empty or show no data

Make sure you have run `fitproj` on all images before running `format` or `analyze`. Also check that image filenames begin with a date in `YYYY_MM_DD` or `MM_DD_YYYY` format.

Quick Reference

Command	What it does
<code>python main.py</code>	Run full pipeline on all images
<code>python main.py match</code>	Template matching, all images
<code>python main.py manual</code>	Manual annotation, all images
<code>python main.py manual --width 1200</code>	Manual annotation with custom window width
<code>python main.py fitproj</code>	Fit & project, all images
<code>python main.py format</code>	Generate plots
<code>python main.py analyze</code>	Interactive plot viewer
<code>python main.py -s 2 -n 1 manual</code>	Manual annotation, image index 2 only
<code>python main.py match fitproj</code>	Match then <code>fitproj</code> , all images

Source code: <https://github.com/ecytryn/Wax-Bite-Image-Processor>